

TAREA PPS05

Usando Docker

Autor: Carlos España Del Pozo - CETI

Contenido

Detalles de la tarea de esta unidad	3
Apartado 1: Manejo básico de contenedores con Docker	3
Respuestas 1	4
Apartado y respuestas 2: Dockerfile	12
Apartado y respuesta 3: Docker-compose	14

Detalles de la tarea de esta unidad

Caso práctico

Julián se ha unido a un grupo de trabajo para la creación de un aplicativo web para entornos web, sabe que sus compañeros están usando Docker y aún tiene cierto temor al uso de esta tecnología por desconocimiento por lo que va repasando cada paso que da.

Esta actividad tiene parte teórica y parte práctica.

Apartado 1: Manejo básico de contenedores con Docker

Comando/línea de comandos	Descripción detallada (si hay opciones se deben explicar)
<code>docker pull mysql:5.7.28</code>	
<code>docker images</code>	
<code>docker run --rm --user mysql -d -p 33060:3306 --name mysql-db1 -e MYSQL_ROOT_PASSWORD=secret -v ./mysql:/var/lib/mysql mysql:5.7.28</code> (NOTA: si el equipo es windows se debe cambiar ./mysql por .\mysql)	
<code>docker ps -a</code>	
<code>docker logs mysql-db1</code>	
<code>docker exec -it mysql-db1 /bin/sh</code>	
<code>docker stop mysql-db1</code>	
<code>docker start mysql-db1</code>	
<code>docker rm -f mysql-db1</code>	

Jenkins es una herramienta de integración continua y entrega continua (CI/CD) de código abierto, escrita en Java, que permite automatizar el desarrollo, prueba y despliegue de software. Visita su página oficial para entenderlo mejor: <https://www.jenkins.io/>

Incluye pantallazos de las pruebas que se piden a continuación, incluyendo las de la consola donde se lanzan las líneas de comandos docker y del navegador con la aplicación.

- 3.1. Lanzar el contenedor de Jenkins (ver <https://github.com/jenkinsci/docker/blob/master/README.md>). Indica la línea de comandos para realizar esta operación.
- 3.2. Comprobar que el contenedor anterior esta funcionando. Indica la línea de comandos para realizar esta operación.

3.3. Mostrar el log del contenedor de Jenkins. Indica la línea de comandos para realizar esta operación.

3.4. Acceder a la aplicación Jenkins desde un navegador (ver <https://www.enkins.io/doc/book/installing/docker/>):

Abre un navegador con la dirección <http://localhost:8080> (si has puesto el 8080)

El token que pide aparece en el log del contenedor.

Escoge la opción de instalar los plugins recomendados.

NO hace falta que crees un usuario (Skip and continue as admin)

3.5. Instalar la tool de Maven con el nombre M3 dentro de Jenkins (Panel de control - Administrar Jenkins - Tools - Instalaciones de Maven - Añadir Maven - Nombre M3 y la opción de instalar automáticamente)

3.6. Crear un pipeline sencillo que se llame XXXXX (donde XXXXX es tu nombre): creas una tarea de tipo pipeline y en el apartado script seleccionas try sample Pipeline... escoges Github+ Maven. Copia el código del pipeline (a ese archivo se le denomina Jenkinsfile) en algún sitio porque te va a hacer falta para responder una pregunta en un apartado posterior

3.7. Ejecutar el pipeline anterior y mostrar el pantallazo de la salida.

3.8. Eliminar el contenedor de Jenkins. Indica la línea de comandos para realizar esta operación.

3.9. Comprobar que el contenedor anterior ya no está funcionando. Indica la línea de comandos para realizar esta operación.

4.1. ¿Qué ventajas aporta trabajar con contenedores?

4.2. Revisa el pipeline que has creado en el apartado anterior ¿Qué hace?

4.3. Busca información sobre pruebas de seguridad en un pipeline e indica que pasos se podrían añadir al pipeline anterior para asegurar que la aplicación que se va a poner en producción es segura

Respuestas 1

1. Primero de todo arrancamos nuestra VM virtual y nos aseguramos de tener el entorno Docker funcionando:

```
docker ps -a
```

```

Fedora 39 CETI [Contenido] - Oracle VM VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

root@fedora:/CETI

Archivo  Editar  Ver  Buscar  Terminal  Ayuda

Drop-In: /usr/lib/systemd/system/service.d
+--10-timeout-abort.conf
Active: active (running) since Sun 2026-03-08 19:19:04 CET; 4min 44s ago
TriggeredBy: + docker.socket
Docs: https://docs.docker.com
Main PID: 1004 (dockerd)
Tasks: 15
Memory: 139.9M
CPU: 1.307s
CGroup: /system.slice/docker.service
+--1004 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

mar 08 19:19:04 fedora dockerd[1004]: time="2026-03-08T19:19:04.657350875+01:00" level=warning msg="WARNING: bridge-nf-call-iptables is deprecated and will be removed from the codebase. Please use bridge-nf-call-iptables."
mar 08 19:19:04 fedora dockerd[1004]: time="2026-03-08T19:19:04.657381971+01:00" level=warning msg="WARNING: bridge-nf-call-iptables is deprecated and will be removed from the codebase. Please use bridge-nf-call-iptables."
mar 08 19:19:04 fedora dockerd[1004]: time="2026-03-08T19:19:04.657397579+01:00" level=info msg="Docker daemon commit-id=978:conf"
mar 08 19:19:04 fedora dockerd[1004]: time="2026-03-08T19:19:04.657531926+01:00" level=info msg="Daemon has completed initialization"
mar 08 19:19:04 fedora dockerd[1004]: time="2026-03-08T19:19:04.862630569+01:00" level=info msg="API listen on /run/docker.sock"
mar 08 19:19:04 fedora systemd[1]: Started docker.service - Docker Application Container Engine.
mar 08 19:22:58 fedora dockerd[1004]: time="2026-03-08T19:22:58.768735520+01:00" level=info msg="Ignoring event" container=b3877486
mar 08 19:22:58 fedora dockerd[1004]: time="2026-03-08T19:22:58.702481891+01:00" level=warning msg="ShouldRestart failed, container=b3877486"
mar 08 19:23:06 fedora dockerd[1004]: time="2026-03-08T19:23:06.454394185+01:00" level=info msg="Ignoring event" container=4e412295
mar 08 19:23:06 fedora dockerd[1004]: time="2026-03-08T19:23:06.474440140+01:00" level=warning msg="ShouldRestart failed, container=4e412295"

root@fedora:/CETI# docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
root@fedora:/CETI#

```

2.

Comando/línea de comandos	Descripción detallada (si hay opciones se deben explicar)
<code>docker pull mysql:5.7.28</code>	Descarga la imagen Docker <code>mysql:5.7.28</code>
<code>docker images</code>	Muestra las imágenes Docker descargadas
<code>docker run --rm --user mysql -d -p 33060:3306 --name mysql-db1 -e MYSQL_ROOT_PASSWORD=secret -v ./mysql:/var/lib/mysql mysql:5.7.28</code> (NOTA: si el equipo es windows se debe cambiar <code>./mysql</code> por <code>.\mysql</code>)	Ejecuta un contenedor Docker con la imagen <code>mysql:5.7.28</code> . Con la opción <code>--name</code> añade el nombre al contenedor, <code>mysql-db1</code> La opción <code>-e</code> establece las variables dentro del contenedor <code>MYSQL_ROOT_PASSWORD= secret</code> La opción <code>-p</code> mapea el puerto del host 33060 en el puerto del contenedor 3306 La opción <code>-v</code> mapea el volumen del host <code>./mysql</code> en el volumen del contenedor <code>/var/lib/mysql</code> La opción <code>--rm</code> establece que una vez parado el contenedor se eliminara.
<code>docker ps -a</code>	Muestra todos los contenedores incluidos los no activos
<code>docker logs mysql-db1</code>	Muestra los logs del contenedor <code>docker logs mysql-db1</code>
<code>docker exec -it mysql-db1 /bin/sh</code>	Ejecuta la bash <code>/bin/sh</code> dentro del contenedor <code>mysql-db1</code> de forma interactiva
<code>docker stop mysql-db1</code>	Para el contenedor <code>mysql-db1</code>
<code>docker start mysql-db1</code>	Arranca el contenedor <code>mysql-db1</code>
<code>docker rm -f mysql-db1</code>	Elimina el contenedor <code>mysql-db1</code>

3. Como vemos en el redame.md del proyecto de github, procedemos a lanzar el contenedor de Docker.



3.1. Lanzar el contenedor de Jenkins

Dentro de nuestra VM ejecutamos el comando:

```
docker run -p 8080:8080 -p 50000:50000 --restart=on-failure jenkins/jenkins:its-jdk21
```

```
root@fedora:~# docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
root@fedora:~# docker run -p 8080:8080 -p 50000:50000 --restart=on-failure jenkins/jenkins:its-jdk21
Unable to find image 'jenkins/jenkins:its-jdk21' locally
its-jdk21: Pulling from jenkins/jenkins
53c88f1df6b7: Pull complete
1354b1a1a1d0: Pull complete
8c1447ae971: Pull complete
3c9c80560818: Pull complete
5651e84d3308: Pull complete
a375846a225: Pull complete
907448ab0206: Pull complete
760b84c2c04: Pull complete
2446a3c0816: Pull complete
f63b11f22c5e: Pull complete
65813881034: Pull complete
7783d6da915: Pull complete
Digest: sha256:23b8ad1806d0e0f2c17288c83ac071127092d726f14b3b844d868468019c7
Status: Downloaded newer image for jenkins/jenkins:its-jdk21
Running from: /usr/share/jenkins/jenkins.war
webroot: /var/jenkins_home/war
2026-03-08 18:43:54.811+0000 [id=1] INFO winstone.Logger$logInternal: Beginning extraction from war file
2026-03-08 18:43:56.282+0000 [id=1] WARNING o.e.j.web.NestedContextHandler$setContextPath: Empty contextPath
2026-03-08 18:43:56.421+0000 [id=1] INFO org.eclipse.jetty.server.Server$doStart: jetty-12.1.5; built: 2025-12-03T22:18:24.732Z
g:1: 4085d0dd7b081e792d7b73946c77b66e4ba25d6; jvm 21.0.9-10-LTS
2026-03-08 18:43:57.103+0000 [id=1] INFO o.e.j.e.w.StandardDescriptorProcessor$visitServlet: NO JSP Support for /, did not find
org.eclipse.jetty.eep.jsp.JspServlet
2026-03-08 18:43:57.258+0000 [id=1] INFO o.e.j.s.DefaultSessionIdManager$doStart: Session workerName=node8
2026-03-08 18:43:57.888+0000 [id=1] INFO hudson.WebAppMain$contextInitialized: Jenkins home directory: /var/jenkins_home found
at: EnvVars.masterEnvVars.get("JENKINS_HOME")
2026-03-08 18:43:57.941+0000 [id=1] INFO o.e.j.s.handler.ContextHandler$doStart: Started oej9m.ContextHandler$CoreContextHandl
er@3651837/Jenkins v2.541.2./D-File:///var/jenkins_home/war,/a=AVAILABLE,h=oej9m.ContextHandler$CoreContextHandler$CoreToNestedHandl
er@450219e/$STARTED()
2026-03-08 18:43:57.954+0000 [id=1] INFO o.e.j.server.AbstractConnector$doStart: Started oejs.ServerConnector@39c1a34e(HTTP/1.1)
```

```
2026-03-08 18:43:58.349+0000 [id=39] INFO jenkins.InitReactorRunner$1onAttained: Listed all plugins
2026-03-08 18:43:59.302+0000 [id=37] INFO jenkins.InitReactorRunner$1onAttained: Prepared all plugins
2026-03-08 18:43:59.399+0000 [id=48] INFO jenkins.InitReactorRunner$1onAttained: Started all plugins
2026-03-08 18:43:59.481+0000 [id=48] INFO jenkins.InitReactorRunner$1onAttained: Augmented all extensions
2026-03-08 18:43:59.717+0000 [id=22] INFO jenkins.InitReactorRunner$1onAttained: System config loaded
2026-03-08 18:43:59.719+0000 [id=38] INFO jenkins.InitReactorRunner$1onAttained: System config adapted
2026-03-08 18:43:59.721+0000 [id=48] INFO jenkins.InitReactorRunner$1onAttained: Loaded all jobs
2026-03-08 18:43:59.724+0000 [id=48] INFO jenkins.InitReactorRunner$1onAttained: Configuration for all jobs updated
2026-03-08 18:43:59.832+0000 [id=54] INFO hudson.util.Retrier$start: Attempt #1 to do the action check updates server
2026-03-08 18:44:00.466+0000 [id=48] INFO jenkins.install.SetupWizard$init:
(LF)>
(LF)> =====
(LF)> =====
(LF)> =====
(LF)> =====
(LF)> Jenkins initial setup is required. An admin user has been created and a password generated.
(LF)> Please use the following password to proceed to installation:
(LF)> #083386d61924211ad6c758372ad78e
(LF)>
(LF)> This may also be found at: /var/jenkins_home/secrets/initialAdminPassword
(LF)>
(LF)> =====
(LF)> =====
(LF)> =====
2026-03-08 18:44:05.144+0000 [id=38] INFO jenkins.InitReactorRunner$1onAttained: Completed initialization
2026-03-08 18:44:05.212+0000 [id=38] INFO hudson.lifecycle.Lifecycle$onReady: Jenkins is fully up and running
2026-03-08 18:44:05.677+0000 [id=54] INFO h.m.DownloadService$DownloadableLoad: Obtained the updated data file for hudson.tasks
Neven MavenInstaller
2026-03-08 18:44:05.679+0000 [id=54] INFO hudson.util.Retrier$start: Performed the action check updates server successfully at t
he attempt #1
```

Vamos a parar y eliminar el contenedor con :

```
docker rm vigilant_hermann
```

Volvemos a ejecutar el contenedor pero esta vez en background para que se quede persistente y con un nombre descriptivo:

```
docker run -d -p 8080:8080 -p 50000:50000 --name jenkins --restart=on-failure jenkins/jenkins:its-jdk21
```

3.2. Comprobamos el contenedor de Jenkins con el comando:

```
docker ps -a
```

```
jenkins@kali:~$ docker run -d --name jenkins --restart=always jenkins/jenkins:its-jdk21
fcc981de0a9f5789a283f22a4fa3d92f48b0c9ffc5a58023fa2acc074b74c83
jenkins@kali:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	NAME	CREATED	STATUS	PORTS
fcc981de0a9f	jenkins/jenkins:its-jdk21	"java/gdn/tini -- /u..."	jenkins	33 seconds ago	Up 32 seconds	0.0.0.0:8080->8080/tcp, 0.0.0.0:50000->50000/tcp

```
jenkins@kali:~$
```

3.3. Mostramos el log del contenedor de Jenkins con el comando:

docker logs jenkins

[illegible]

3.4. Accedemos a la aplicación Jenkins desde un navegador:

http://localhost:8080

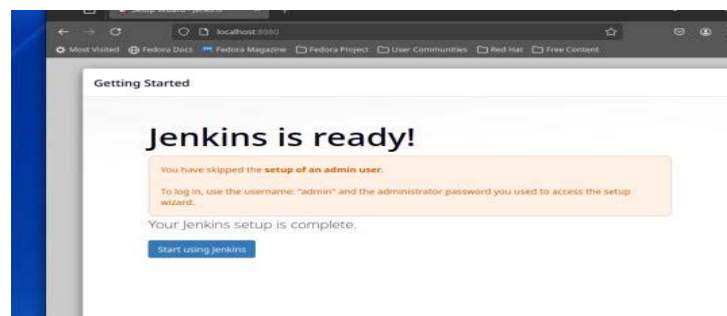
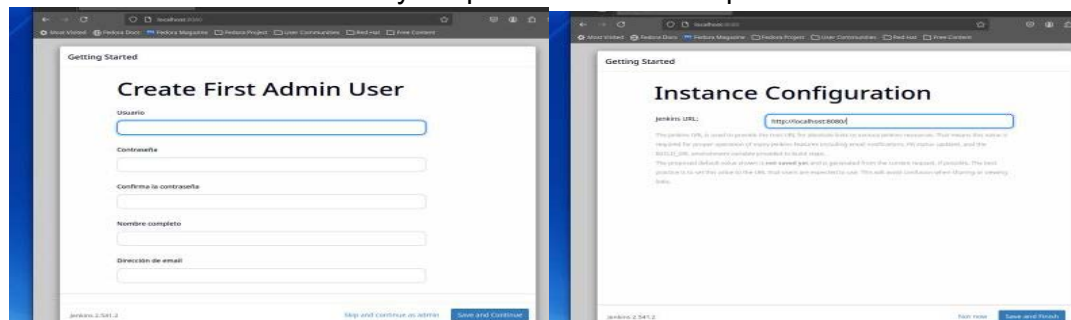


Añadimos el TOKEN de nuestra instalación y pinchamos en continuar:

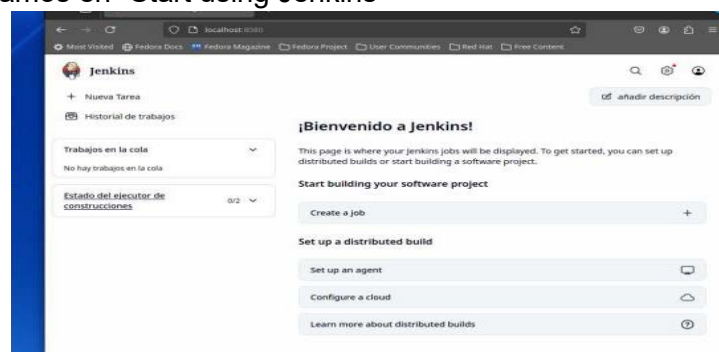
```
[LF]> Jenkins initial setup is required. An admin user has been created and a password generated.  
[LF]> Please use the following password to proceed to installation:  
[LF]>  
[LF]> a133968f83454caa51a16c39c236221  
[LF]>  
[LF]> This may also be found at: /var/jenkins_home/secrets/initialAdminPassword  
[LF]>  
[LF]> .....  
[LF]> .....  
[LF]> .....
```



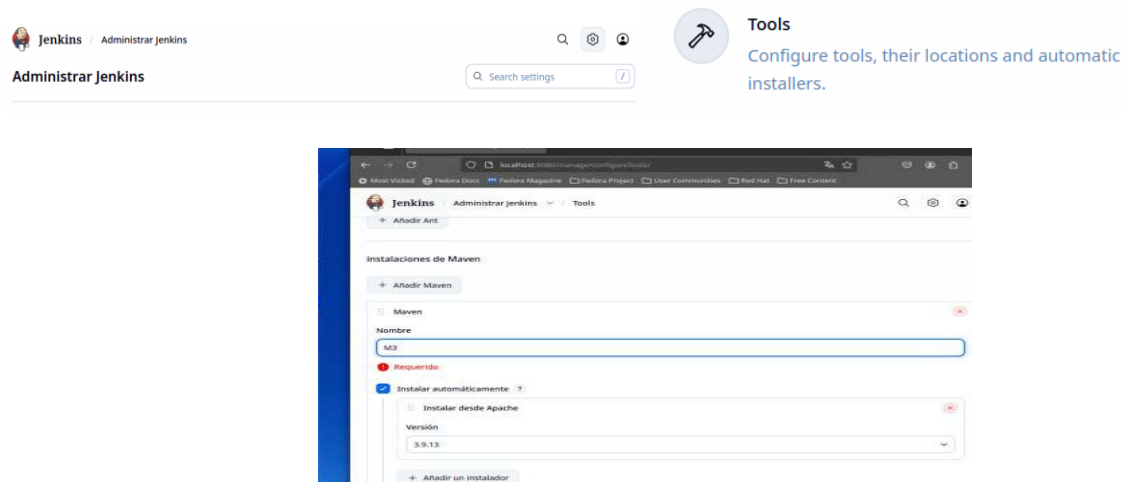

Continuamos como administrador y aceptamos la instancia por defecto



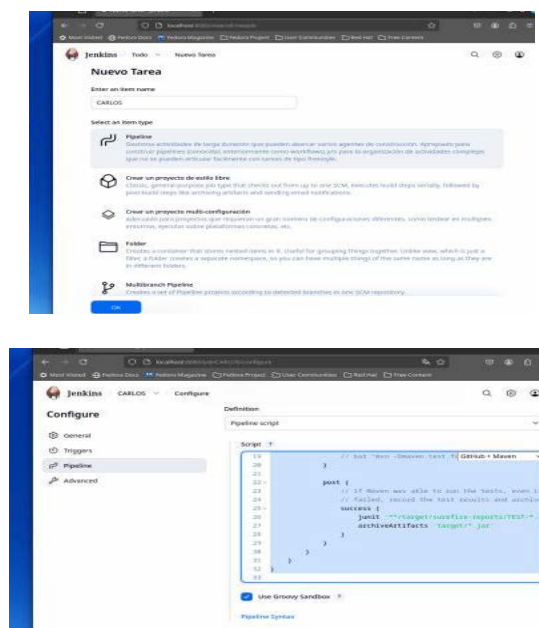
Finalmente pinchamos en “Start using Jenkins”



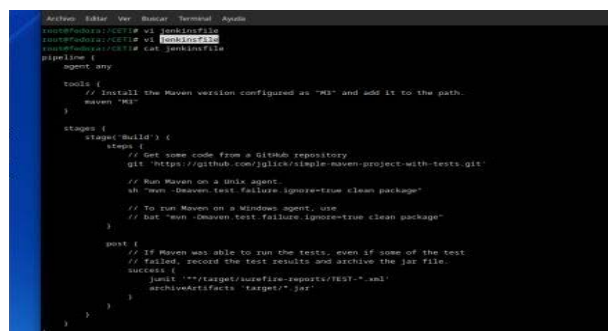
3.5. Instalamos la tool de Maven con el nombre M3 dentro de Jenkins:



3.6. Creamos un pipeline sencillo que se llame: CARLOS



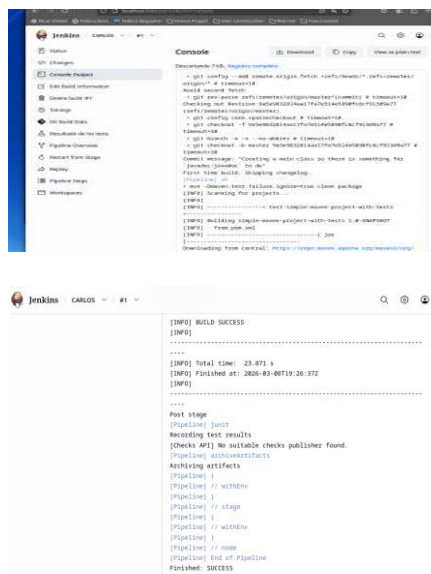
Guardamos el pipeline en un fichero en la ruta /CETI/jenkinsfile



3.7. Ejecutamos el pipeline anterior pinchando en “construir ahora”:

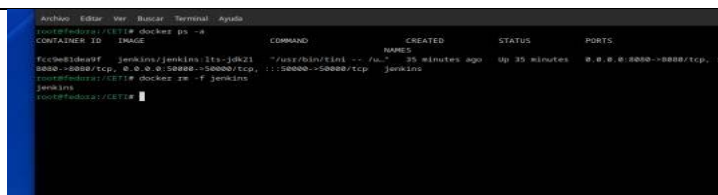


Vemos el resultado en “Console Output”



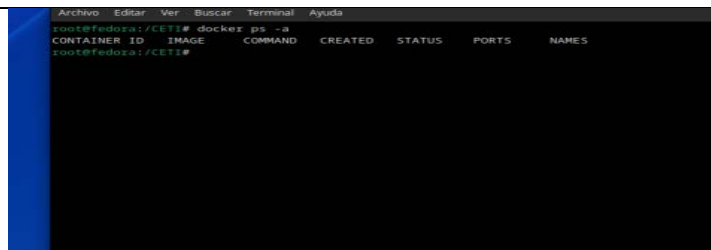
3.8. Eliminamos el contenedor de Jenkins con el siguiente comando:

```
docker rm -f jenkins
```



3.9. Comprobamos que el contenedor de Jenkins esté eliminado con el siguiente comando:

```
docker ps -a
```



4.1. ¿Qué ventajas aporta trabajar con contenedores?

Trabajar con contenedores ofrece grandes ventajas, sobre todo a la hora de desplegar imágenes y aplicaciones portables que no dependan del Sistema operativo en el que instalamos.

también conseguimos aislar cada contenedor siendo más seguros.

Finalmente he de indicar que la rapidez de despliegue lo hace ideal para entornos CI/CD y muy escalable.

4.2. Revisa el pipeline que has creado en el apartado anterior ¿Qué hace?

El pipeline implementa un proceso básico de integración continua.

Descarga un proyecto de github de prueba de Maven.

Compila dicho proyecto con Maven y finalmente ejecuta un test de pruebas unitarias para verificar que funciona correctamente.

4.3. Busca información sobre pruebas de seguridad en un pipeline e indica que pasos se podrían añadir al pipeline anterior para asegurar que la aplicación que se va a poner en producción es segura

Podemos incluir pruebas de seguridad automáticas antes de desplegar la aplicación.

Podemos utilizar herramientas como “SonarQube” para detectar problemas del código fuente.

Tambien podemos analizar la imagen Docker antes de desplegarla con herramientas como Trivy.

Un ejemplo con Sonarqube sería:

```
pipeline {
  agent any

  tools {
    maven "M3"
  }

  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/jglick/simple-maven-project-with-tests.git'
      }
    }

    stage('Build') {
      steps {
        sh "mvn -Dmaven.test.failure.ignore=true clean package"
      }
    }
  }
}
```

```

    post {
        success {
            junit '**/target/surefire-reports/TEST-*.xml'
            archiveArtifacts 'target/*.jar'
        }
    }
}

stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQubeServer') {
            sh "mvn sonar:sonar"
        }
    }
}

stage('Quality Gate') {
    steps {
        timeout(time: 2, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
        }
    }
}
}
}

```

Apartado y respuestas 2: Dockerfile

1. Dado el siguiente archivo Dockerfile:

```

FROM ubuntu
LABEL maintainer="admin@ejemplo.com"
ARG APP_DATO1=1
RUN apt-get update && apt-get install -y apache2
RUN mkdir /var/www/html/ejemplo
EXPOSE 80
ADD index2.html /var/www/html/
ENTRYPOINT ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]

```

FROM ubuntu	Indica que la imagen base del contenedor será Ubuntu.
LABEL maintainer="admin@ejemplo.com"	Añade información adicional de la imagen. (informativa)
ARG APP_DATO1=1	Define la variable APP_DATO1=1 que se puede usar durante la construcción de la imagen.
RUN apt-get update && apt-get install -y apache2	Ejecuta en la construcción de la imagen los comandos indicados concatenados, actualiza los repositorios e instala el servidor web Apache
EXPOSE 80	Indica que el contenedor usará el puerto 80 (informativo)

ADD index2.html /var/www/html/	Copia el archivo index2.html desde el host al contenedor a la ruta /var/www/html/
ENTRYPOINT ["usr/sbin/apache2ctl", "-D", "FOREGROUND"]	Define el comando que se ejecuta al iniciar el contenedor (arranca Apache).

Responde a las siguientes preguntas y realiza las pruebas

- ## 2. ¿Para qué es útil un archivo Dockerfile?

Un Dockerfile se usa para crear imágenes docker

3. ¿Con qué comando se crean las imágenes partiendo de un Dockerfile?

El comando a ejecutar es variable, pero para definir un nombre de etiqueta de la imagen sería:

```
docker build -t apache:1.0 .
```

4. Realiza una prueba con el archivo anterior:

Creamos el archivo Dockerfile:

```

Archivo  Editar  Ver  Buscar  Terminal  Ayuda
root@fedora:/CETI# vi Dockerfile
root@fedora:/CETI# cat Dockerfile
FROM ubuntu
LABEL maintainer="admin@ejemplo.com"
ARG APP_DATO1=1
RUN apt-get update && apt-get install -y apache2
RUN mkdir /var/www/html/ejemplo
EXPOSE 80
ADD index2.html /var/www/html/
ENTRYPOINT ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]
root@fedora:/CETI#

```

Ejecutamos el comando de construcción de la imagen y nos da error por no tener el archivo index2.html

[illegible]

- #### 4.1. Crea un directorio

```
mkdir apache
```

- #### 4.2. Crea dentro del directorio el archivo Dockerfile anterior

```
mkdir apache
```

4.3. Crea el archivo index2.html (escribe dentro tu nombre)

```
echo "CARLOS" > index2.html
```

4.4. Crea una imagen nueva con ese archivo Dockerfile de nombre mi-apache y versión 1.0

```
docker build -t mi-apache:1.0 .
```

[illegible]

4.5. Lanza un contenedor con la imagen anterior en el puerto 7777.

```
docker run --name mi-apache -p 7777:80 mi-apache:1.0
```

```
root@kali:~/Desktop/CTF/Reverse-Engineering# docker run --name mi-apache -p 9222:80 mi-apache:1.0
root@kali:~/Desktop/CTF/Reverse-Engineering# docker run --name mi-apache -p 9222:80 mi-apache:1.0
Aaaa555: search2: Could not globally determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName'
directive globally to suppress this message
```

4.6. Abre un navegador y accede al contenedor anterior mostrando las páginas index.html e index2.html

[illegible]

Apartado y respuesta 3: Docker-compose

línea de comandos: docker-compose up

Ejecuta los servicios (contenedores, volúmenes, redes) que estén definidos en Docker-compose.yml

línea de comandos: docker-compose down	Detiene y elimina los contenedores creados en el Docker compose
services	Define los contenedores, redes y volúmenes que se van a ejecutar en el docker-compose
image	Indica la imagen Docker que se usará para cada contenedor
build	Permite crear una construcción de una imagen desde un Dockerfile
container-name	El nombre que se dará a cada contenedor definido en el archivo Docker-compose
ports	Los puertos o mapeos a utilizar por cada contenedor
volumes	Los volúmenes o mapeos de volúmenes a utilizar por cada contenedor
environment	Las variables de entorno a utilizar por cada contenedor normalmente definidas en el .env

3. Realiza las siguientes pruebas (incluye pantallazos de cada una)

3.1. Lanza dos contenedores con las siguientes líneas en una terminal y muestra pantallazos de la ejecución de esos comandos (docker ps -a)

```
docker run -d -p 33060:3306 --name mysql-db1 -e MYSQL_ROOT_PASSWORD=secret -v mysql:/var/lib/mysql mysql:5.7.28
docker run -d --rm --name phpmyadmin --link mysql-db1 -e PMA_HOST=mysql-db1 -p 9080:80 phpmyadmin/phpmyadmin
```

Ejecutamos los comandos siguientes:

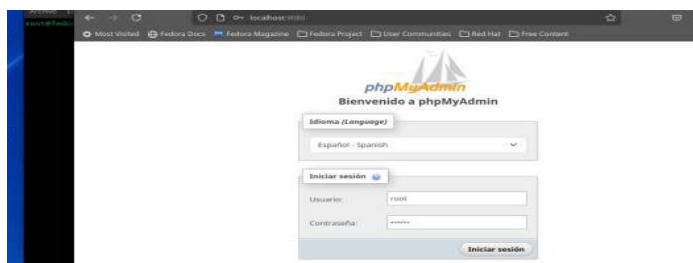
```
root@fedora:~/CET1/apache# docker run -d -p 33060:3306 --name mysql-db1 -e MYSQL_ROOT_PASSWORD=secret -v mysql:/var/lib/mysql mysql:5.7.28
Unable to find image 'mysql:5.7.28' locally
5.7.28: Pulling from library/mysql
884555ee8376: Pull complete
c53bda48b794: Pull complete
ca9d7277f79e: Pull complete
2d7aad6c590e: Pull complete
8d6ca53c7f9e: Pull complete
8ddae896e768: Pull complete
327aed7b6e7b: Pull complete
31f1f838b027: Pull complete
a5a3ad97e819: Pull complete
48bede7828ac: Pull complete
388afa2ee972: Pull complete
Digest: sha256:b38555e9338edf225dea22aeb104eed79fc8bd2f064fde16e1804d0dd8fc
Status: Downloaded newer image for mysql:5.7.28
04bde49b4edebd3329276c2baf7d7211e352e2ebab106655928cb847898
root@fedora:~/CET1/apache#
```

```
root@fedora:~/CET1/apache# docker run -d --rm --name phpmyadmin --link mysql-db1 -e PMA_HOST=mysql-db1 -p 9080:80 phpmyadmin/phpmyadmin
Unable to find image 'phpmyadmin/phpmyadmin:latest' locally
latest: Pulling from phpmyadmin/phpmyadmin
6e77382f187: Pull complete
244e2a1f6855: Pull complete
e1f7e4e0d179: Pull complete
2480da88b67: Pull complete
43822f7d0e1: Pull complete
0453c1f0b74c: Pull complete
43822f7d0e1: Pull complete
6571f0b0b02: Pull complete
8082c96b0a0a: Pull complete
740b2ca09005: Pull complete
4ee4d54c28c: Pull complete
00a778e422f8: Pull complete
00a778e422f8: Pull complete
47a70708e754: Pull complete
2728212726a: Pull complete
a8c81cc8b0d0: Pull complete
2728212726a: Pull complete
00a778e422f8: Pull complete
05a243c105e: Pull complete
Digest: sha256:42a2880670ae79f9f32c59ade221cf039906e54b00adca8e6e756d0fbb
Status: Downloaded newer image for phpmyadmin/phpmyadmin:latest
87edebe8da81318ef0adcc40823fa8e123554c447feba0d0771aaf1e36c
root@fedora:~/CET1/apache# docker ps -a
```

CONTAINER ID	IMAGE	NAME	COMMAND	CREATED	STATUS	PORTS
67ade480dad	phpmyadmin/phpmyadmin		"/docker-entrypoint..."	0 seconds ago	Up 7 seconds	0.0.0.0:9080->80/tcp, [::]:9080->80/tcp
04bde49b4ed	mysql:5.7.28		"docker-entrypoint.s..."	About a minute ago	Up About a minute	33060/tcp, 0.0.0.0:33060->3306/tcp, [::]:33060->3306/tcp
228157a7195	al-seoche:1.0	al-seoche	"./usr/bin/apache2c..."	12 minutes ago	Up 12 minutes	0.0.0.0:7777->80/tcp, [::]:7777->80/tcp

- 3.2. Utilizando el contenedor de phpmyadmin crea una base de datos con tu nombre en mysql. Muestra pantallazos del navegador con el acceso a phpmyadmin y con la creación de la base de datos

Abrimos un navegador y accedemos a: <http://localhost:9080/>:



Pinchamos en Nueva -> Crear base de datos



- 3.3. Crea un archivo **docker-compose.yml** que permita lanzar esos dos contenedores utilizando docker compose. Incluye el contenido del archivo docker-compose.yml

Creamos el siguiente archivo **docker-compose.yml**

```

services:
  mysql-db1:
    image: mysql:5.7.28
    container_name: mysql-db1
    environment:
      MYSQL_ROOT_PASSWORD: secret
    ports:
      - "33060:3306"
    volumes:
      - mysql:/var/lib/mysql

  phpmyadmin:
    image: phpmyadmin/phpmyadmin
  
```

```
container_name: phpmyadminc
environment:
  PMA_HOST: mysql-db1
ports:
  - "9080:80"
depends_on:
  - mysql-db1
volumes:
mysql:
```

3.4. Lanza el archivo anterior. Muestra pantallazos de la ejecución en la consola

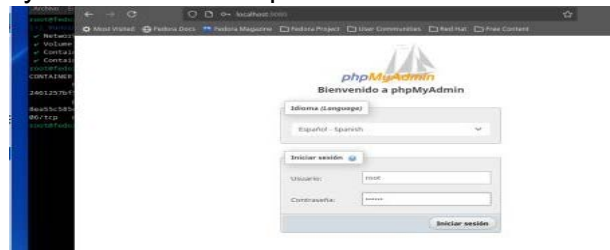
Lo ejecutamos con el comando:

```
docker compose up -d
```

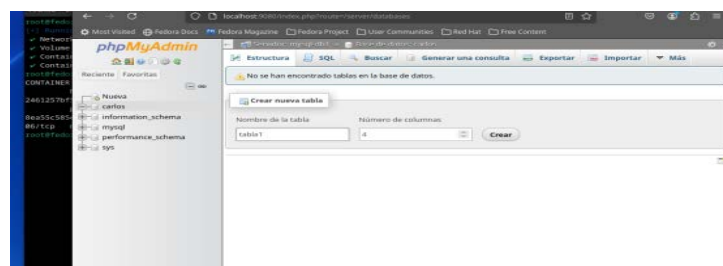
```
root@fedora:~/C171# docker compose up -d
[+] Building 4.0s
Network c171_default Created
Volume "c171_mysql" Created
Container mysql-db1 Created
Container phpmyadminc Created
root@fedora:~/C171# docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS
2401257b7543   phpmyadmin/phpmyad  "/docker-entrypoint..." 7 seconds ago  up 5 seconds  0.0.0.0:9080->80/tcp, [::]:9080->80/tcp
8a955c585ad5   mysql:5.7.28        "docker-entrypoint s..." 7 seconds ago  up 6 seconds  33060/tcp, 0.0.0.0:33060->33060/tcp, [::]:33060->33060/tcp
```

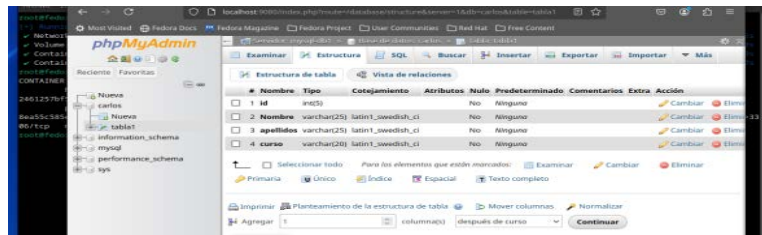
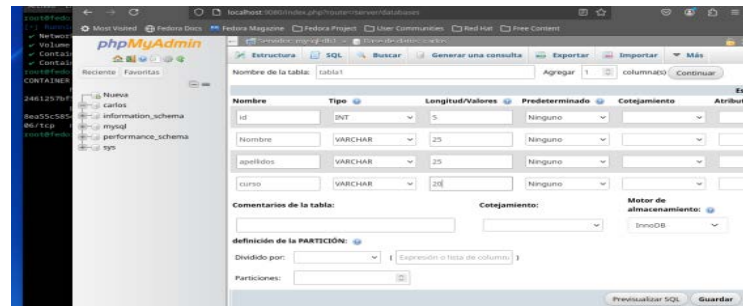
3.5. Utilizando el contenedor de phpmyadmin crea una tabla en la base de datos. Muestra pantallazos del navegador con el acceso a phpmyadmin y con la creación de la tabla

Abrimos un navegador y accedemos a: <http://localhost:9080/>:



Pinchamos en Crear nueva tabla -> añadimos los cuatro campos





3.6. Termina los contenedores lanzados. Muestra pantallazos de la ejecución en la consola

Paramos los contenedores ejecutando el comando:

docker compose down

